

## Praktikum 8: Kürzeste Wege

### Aufgabe 1: Dijkstra – Reindeer Maze

Das Rätsel „Advent of Code 2024, Tag 16“ (<https://adventofcode.com/2024/day/16>) beschreibt einen Rentierparcours: Ein Rentier startet an Position S mit Blickrichtung Ost. Es kann sich in einem Schritt vorwärts bewegen (Kosten: 1) oder um 90° drehen (Kosten: 1000). Gesucht ist der günstigste Weg von S zum Zielfeld E.

#### Das Beispiel-Labyrinth:

```
#####
# .....# .....E#
# .# .### .# .### .#
# .....# .# .....#
# .### .##### .# .#
# .# .# .....# .#
# .# .##### .### .#
# .....# .....# .#
### .# .##### .# .#
# ...# .....# .# .#
# .# .# .### .# .# .#
# .....# .....# .# .#
# .### .# .# .# .# .#
# S .# .....# .....#
#####
```

- Modellierung:** Ein Zustand ist ein Tripel (Zeile, Spalte, Richtung) mit Richtung  $\in \{N, O, S, W\}$ . Eine Kante existiert beim Vorwärtsbewegen (Kosten 1) und beim Drehen (Kosten 1000). Wie viele Knoten und Kanten hat der vollständige Graph für eine Karte mit R Zeilen und C Spalten (ohne Wände)? Welche Darstellung (Adjazenzliste oder -matrix) ist geeigneter?
- Dijkstra:** Bauen Sie den Graphen auf und nutzen Sie `dijkstra()` aus `vorlesung/L08_graphen/graph.py`, um den günstigsten Weg von S (Startrichtung Ost) nach E zu finden. Geben Sie die Gesamtkosten und den Pfad aus.
- AoC:** Melden Sie sich bei „Advent of Code“ an und laden Sie Ihren eigenen Parcours herunter. Ermitteln Sie die minimalen Kosten.

## Aufgabe 2: Bellman-Ford

Der Bellman-Ford-Algorithmus findet kürzeste Wege auch bei negativen Kantengewichten und erkennt negative Zyklen. Der folgende Graph entspricht dem Vorlesungsbeispiel:

| Von | Nach | Gewicht |
|-----|------|---------|
| a   | b    | 9       |
| a   | d    | 4       |
| b   | c    | 1       |
| b   | d    | 2       |
| c   | b    | 5       |
| c   | e    | 4       |
| d   | c    | -2      |
| d   | e    | 2       |

- Implementierung:** Implementieren Sie eine Funktion `bellman_ford(graph, start_name)` analog zu `dijkstra()`, die einen beliebigen Graphen (Unterklasse von `Graph`) als Parameter entgegennimmt. Nutzen Sie `graph.all_vertices()` und `graph.all_edges()` für den Zugriff auf den Graphen. Die Funktion gibt ein Tupel (`distance_map`, `predecessor_map`, `has_negative_cycle`) zurück. Orientieren Sie sich am Algorithmus aus der Vorlesung:  $|V| - 1$  Durchläufe über alle Kanten zur Relaxation, danach ein abschließender Prüfdurchlauf zur Erkennung negativer Zyklen.
- Test:** Bauen Sie den Graphen aus der Tabelle auf und führen Sie Bellman-Ford von Knoten **a** aus. Geben Sie für jeden Knoten die kürzeste Distanz und den vollständigen Pfad aus.
- Negativer Zyklus:** Fügen Sie die Kante **e** → **b** mit Gewicht **-8** hinzu. Was gibt `has_negative_cycle` zurück? Erläutern Sie, welche Prüfung im Algorithmus den Zyklus erkennt, und zeigen Sie den betroffenen Zyklus im Graphen.
- Komplexität:** Bellman-Ford hat die Zeitkomplexität  $O(|V| \cdot |E|)$ , Dijkstra mit Min-Heap  $O(|V| \log |V| + |E|)$ . Wann ist Dijkstra vorzuziehen, und wann muss Bellman-Ford eingesetzt werden?